

使用 OpenTelemetry 檢測追蹤資訊

來源：<https://codelabs.developers.google.com/codelabs/otel-cloudtrace?hl=zh-tw>

步驟數：6

OpenTelemetry 是業界觀測追蹤記錄和指標的業界標準。這場講座介紹瞭如何使用 OpenTelemetry 檢測應用程式指標，以及如何在 Cloud Monitoring 和其他監控工具中運用這些指標。

目錄

1. [1. 簡介](#)
2. [2. 設定和需求](#)
3. [3. 建構、推送及部署微服務](#)
4. [4. HTTP 的檢測](#)
5. [5. gRPC 的檢測](#)
6. [6. 恭喜](#)

1. 簡介



上次更新時間：2021 年 3 月 5 日

應用程式的觀測能力

觀測能力和 OpenTelemetry

「可觀測性」一詞用於描述系統的屬性。如果系統具備可觀測性，團隊就能主動偵錯。在這種情況下，可觀測性的三大支柱 (記錄、指標和追蹤記錄) 是系統取得可觀測性的基本工具。

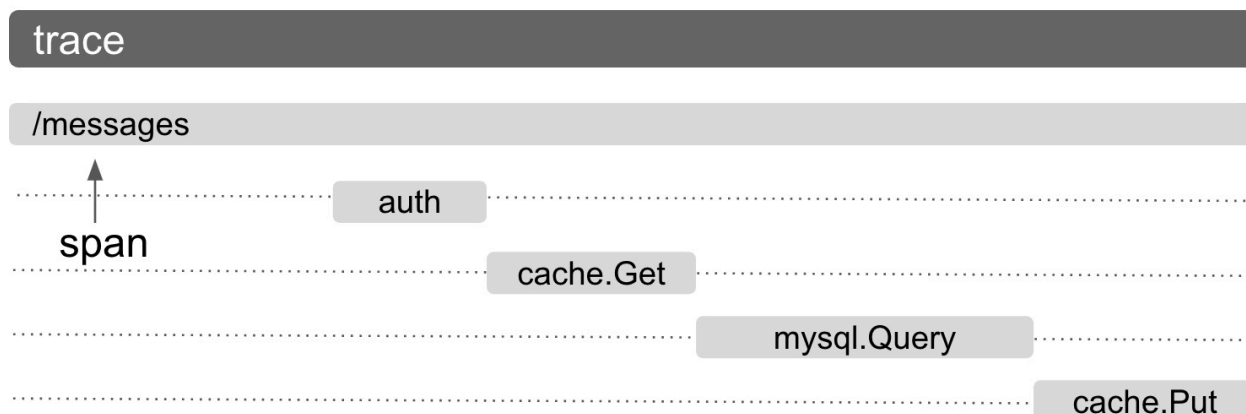
OpenTelemetry 是一組規格和 SDK，可加速遙測資料 (記錄、指標和追蹤記錄) 的檢測和匯出作業，滿足可觀測性需求。OpenTelemetry 是 CNCF 旗下由社群主導的開放標準專案。開發人員可以運用專案及其生態系統提供的程式庫，以廠商中立的方式針對多種架構檢測應用程式。

分散式追蹤

在記錄、指標和追蹤記錄中，追蹤記錄是遙測資料，可顯示系統中特定程序部分的延遲時間。尤其在微服務時代，分散式追蹤是找出整體分散式系統延遲瓶頸的強大推手。

分析分散式追蹤記錄時，追蹤記錄資料的視覺化呈現方式是掌握整體系統延遲時間的關鍵。在分散式追蹤中，我們會處理一組呼叫，以處理系統進入點的單一要求，形式為包含多個 Span 的追蹤。

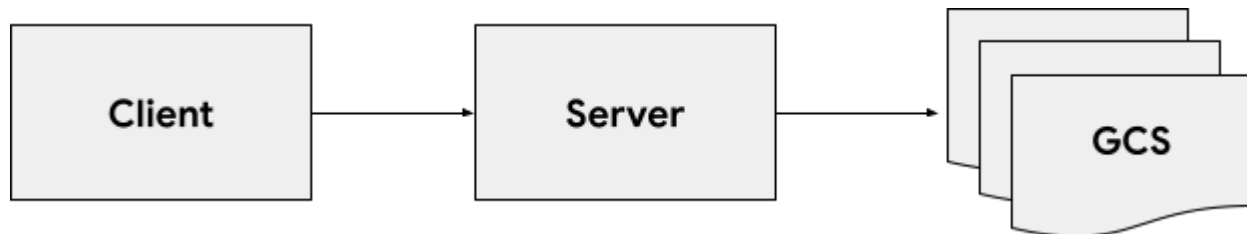
時距代表分散式系統中完成的個別工作單元，會記錄開始和停止時間。跨度之間通常具有階層關係，如下圖所示，所有較小的跨度都是大型 /messages 跨度的子項跨度，並組合成一個追蹤記錄，顯示工作在系統中的路徑。



Google Cloud Trace 是分散式追蹤後端的選項之一，且與 Google Cloud 中的其他產品整合良好。

建構項目

在本程式碼研究室中，您將在名為「Shakesapp」的服務中，檢測追蹤記錄資訊。這項服務會在 Google Kubernetes Engine 執行的 Kubernetes 叢集上運作。Shakesapp 的架構如下所述：



- 用戶端將查詢字串傳送至伺服器
- 伺服器會接受用戶端的查詢，從 Google Cloud Storage 擷取所有莎士比亞作品的文字格式，搜尋包含查詢的行，並將相符的行數傳回給用戶端。

您會在要求中記錄追蹤資訊。

課程內容

- 如何在 Python 專案中開始使用 OpenTelemetry Trace 程式庫
- 如何使用程式庫建立範圍
- 如何在應用程式元件之間的線路傳播範圍內容
- 如何將追蹤記錄資料傳送至 Google Cloud Trace
- 如何在 Google Cloud Trace 中分析追蹤記錄

本程式碼研究室說明如何檢測微服務。為方便瞭解，這個範例只包含 3 個元件 (負載產生器、用戶端和伺服器)，但您可以將本程式碼研究室中說明的相同程序，套用至更複雜的大型系統。

軟硬體需求

- Python 3 知識
-

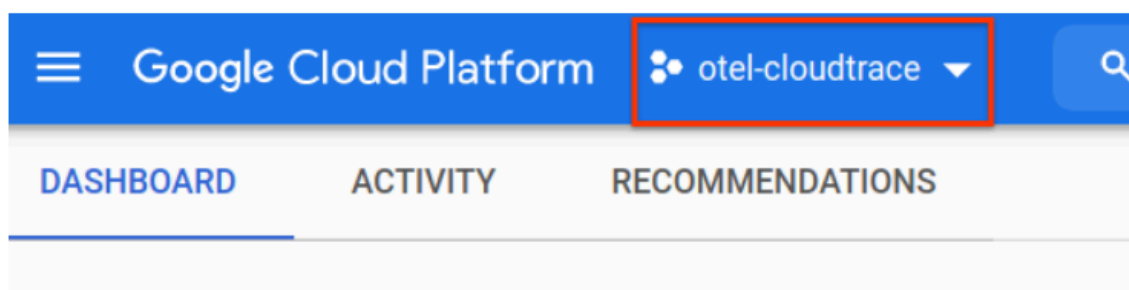
步驟 2 · 約 0 分鐘

2. 設定和需求

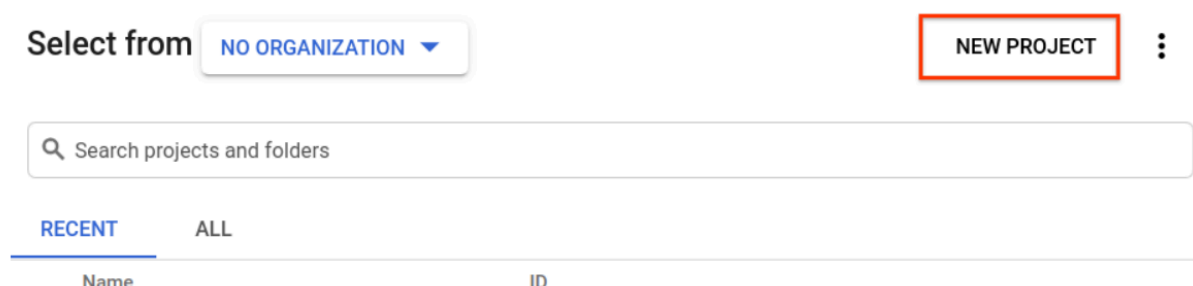
自修實驗室環境設定

如果您沒有 Google 帳戶 (Gmail 或 Google 應用程式)，請先[建立帳戶](#)。登入 Google Cloud Platform 主控台 (console.cloud.google.com)，然後建立新專案。

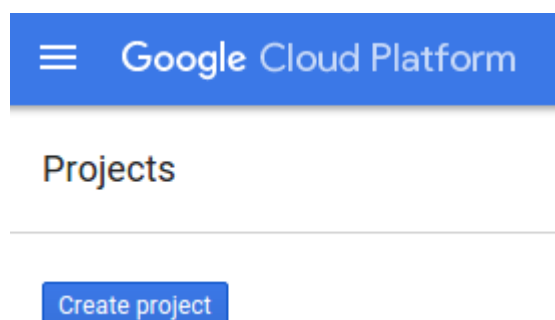
如果您已有專案，請按一下主控台左上方的專案選取下拉式選單：



然後在隨即顯示的對話方塊中，按一下「NEW PROJECT」(新專案) 按鈕，建立新專案：



如果您還沒有專案，應該會看到如下對話方塊，請建立第一個專案：



後續的專案建立對話方塊會顯示，讓您輸入新專案的詳細資料：

New Project

Project name *

otel-cloudtrace



Project ID: otel-cloudtrace-306615. It cannot be changed later. [EDIT](#)

Billing account *



Any charges for this project will be billed to the account you select here.

Organization *

請記住專案 ID，所有 Google Cloud 專案的專案 ID 都是不重複的名稱 (上述名稱已遭占用，因此不適用於您，抱歉！)。本程式碼研究室稍後會將其稱為 PROJECT_ID。

接下來，如果尚未啟用，您需要在開發人員控制台中[啟用帳單](#)，才能使用 Google Cloud 資源並[啟用 Cloud Trace API](#)。

Billing

This project has no billing account

This project is not linked to a billing account

[Link a billing account](#)

or

[Manage billing accounts](#)

完成本程式碼研究室的費用不應超過數美元，但如果您決定使用更多資源，或是將資源繼續執行 (請參閱本文件結尾的「清除」一節)，則可能會增加費用。Google Cloud Trace、Google Kubernetes Engine 和 Google Artifact Registry 的價格均列於官方說明文件中。

- [Google Cloud Observability 定價](#)
- [定價 | Kubernetes Engine 說明文件](#)
- [Artifact Registry 定價 | Artifact Registry 說明文件](#)

Google Cloud Platform 新使用者享有價值 **\$300 美元的免費試用期**，因此本程式碼研究室應完全免費。

Google Cloud Shell 設定

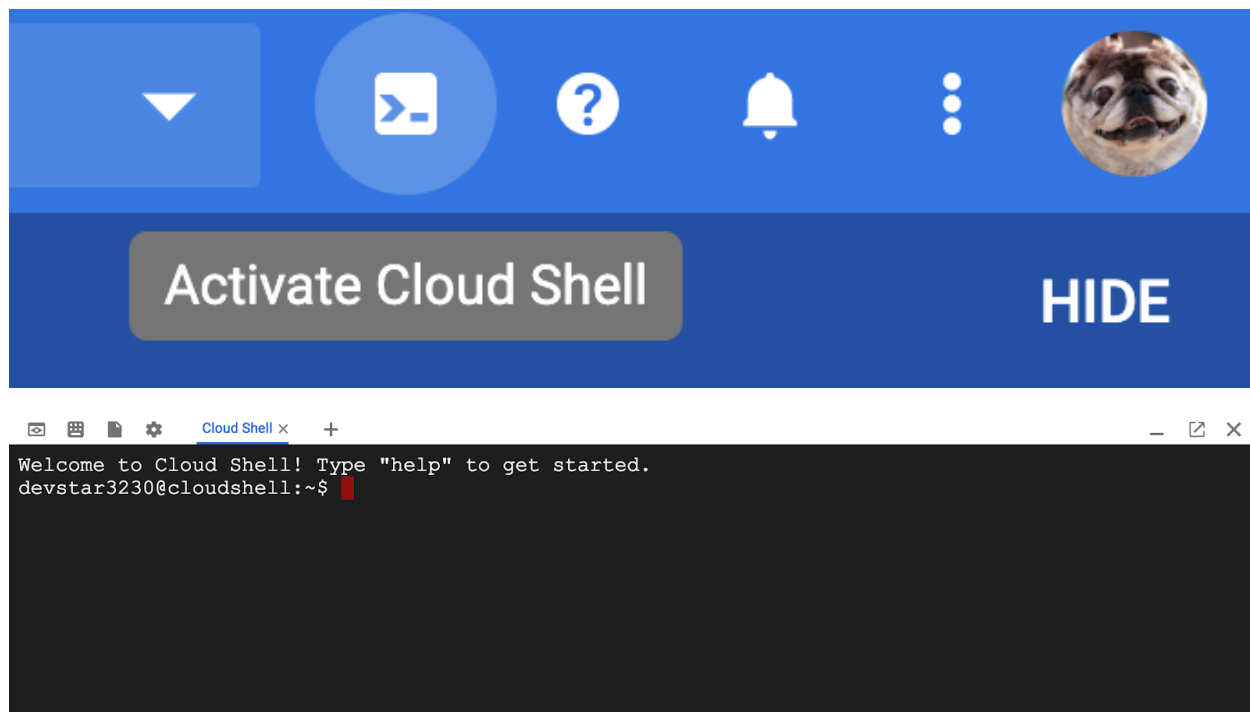
雖然可以透過筆電遠端操作 Google Cloud 和 Google Cloud Trace，但在本程式碼研究室中，我們將使用 [Google Cloud Shell](#)，這是 Cloud 中執行的指令列環境。

這部以 Debian 為基礎的虛擬機器，搭載各種您需要的開發工具，並提供永久的 5GB 主目錄，而且可在 Google Cloud 運作，大幅提升網路效能並強化驗證功能。也就是說，您只需要瀏覽器 (Chromebook 也可以) 就能完成本程式碼研究室。

如要從 Cloud 控制台啟用 Cloud Shell，只要按一下「啟用 Cloud Shell」圖示



即可 (佈建並連線至環境的作業需要一些時間才能完成)。



連至 Cloud Shell 後，您應該會看到驗證已完成，專案也已設為獲派的專案 ID `PROJECT_ID`。

```
gcloud auth list
```

指令輸出

```
Credentialed accounts:
- <myaccount>@<mydomain>.com (active)
```

注意：Cloud Shell 已預先安裝 `gcloud` 指令列工具，並支援 Tab 鍵完成功能，相信您一定會喜歡。詳情請參閱 [gcloud 指令列工具總覽](#)。

```
gcloud config list project
```

指令輸出

```
[core]
project = <PROJECT_ID>
```

如果專案未設定，請發出下列指令：

```
gcloud config set project <PROJECT_ID>
```

在尋找「`PROJECT_ID`」嗎？請檢查您在設定步驟中使用的 ID，或在 Cloud 控制台資訊主頁中尋找：

 Google Cloud Platform 

DASHBOARD

ACTIVITY

RECOMMENDATIONS

 **Project info** 

Project name

my-project

Project ID

coral-bucksaw-280805

Project number

58646390136

[ADD PEOPLE TO THIS PROJECT](#)

[→ Go to project settings](#)

Cloud Shell 也會預設設定部分環境變數，這些變數在您執行後續指令時可能很有用。

```
echo $GOOGLE_CLOUD_PROJECT
```

指令輸出

```
<PROJECT_ID>
```

最後，設定預設可用區和專案。

```
gcloud config set compute/zone us-central1-f
```

你可以選擇各種不同區域。詳情請參閱「[地區和區域](#)」。

注意：如果在自己的機器上執行 `gcloud`，各個工作階段都會保留這些設定。但在 Cloud Shell 中，每次啟動新工作階段或重新連線時，都必須重新設定。

Python 設定

在本程式碼研究室中，我們會使用「poetry」嚴格管理套件版本。在 Cloud Shell 中執行下列指令：

```
curl -sSL https://raw.githubusercontent.com/python-poetry/poetry/master/get-poetry.py  
| python3 -  
source $HOME/.poetry/env
```

設定 Google Kubernetes 叢集

在本程式碼研究室中，您會在 Google Kubernetes Engine (GKE) 上執行微服務叢集。本程式碼研究室的流程如下：

1. 將基準專案下載至 Cloud Shell
2. 將微服務建構到容器中
3. 將容器上傳至 Google Artifact Registry (GAR)
4. 將容器部署至 GKE
5. 修改服務的原始碼，以便進行追蹤檢測
6. 前往步驟 2

啟用 Kubernetes Engine

首先，我們要設定 Kubernetes 叢集，讓 Shakesapp 在 GKE 上執行，因此需要啟用 GKE。前往「Kubernetes Engine」選單，然後按下「啟用」按鈕。



Kubernetes Engine API

Google

Builds and manages container-based applications, powered by the open source Kubernetes technology.

ENABLE

TRY THIS API [↗](#)

現在可以開始建立 Kubernetes 叢集。

建立 Kubernetes 叢集

在 Cloud Shell 執行下列指令，建立 Kubernetes 叢集。請確認可用區值位於您用於建立 Artifact Registry 存放區的區域下方。如果存放區區域未涵蓋該區域，請變更區域值 `us-central1-f`。

```
gcloud container clusters create otel-trace-codelab --zone us-central1-f \  
--num-nodes 1 \  
--machine-type e2-highcpu-4
```

指令輸出

```
Creating cluster otel-trace-codelab in us-central1-f... Cluster is being health-
checked (master is healthy)...done.
Created [https://container.googleapis.com/v1/projects/psychic-order-307806/zones/us-
central1-f/clusters/otel-trace-codelab].
To inspect the contents of your cluster, go to:
https://console.cloud.google.com/kubernetes/workload_/gcloud/us-central1-f/otel-
trace-codelab?project=psychic-order-307806
kubeconfig entry generated for otel-trace-codelab.
```

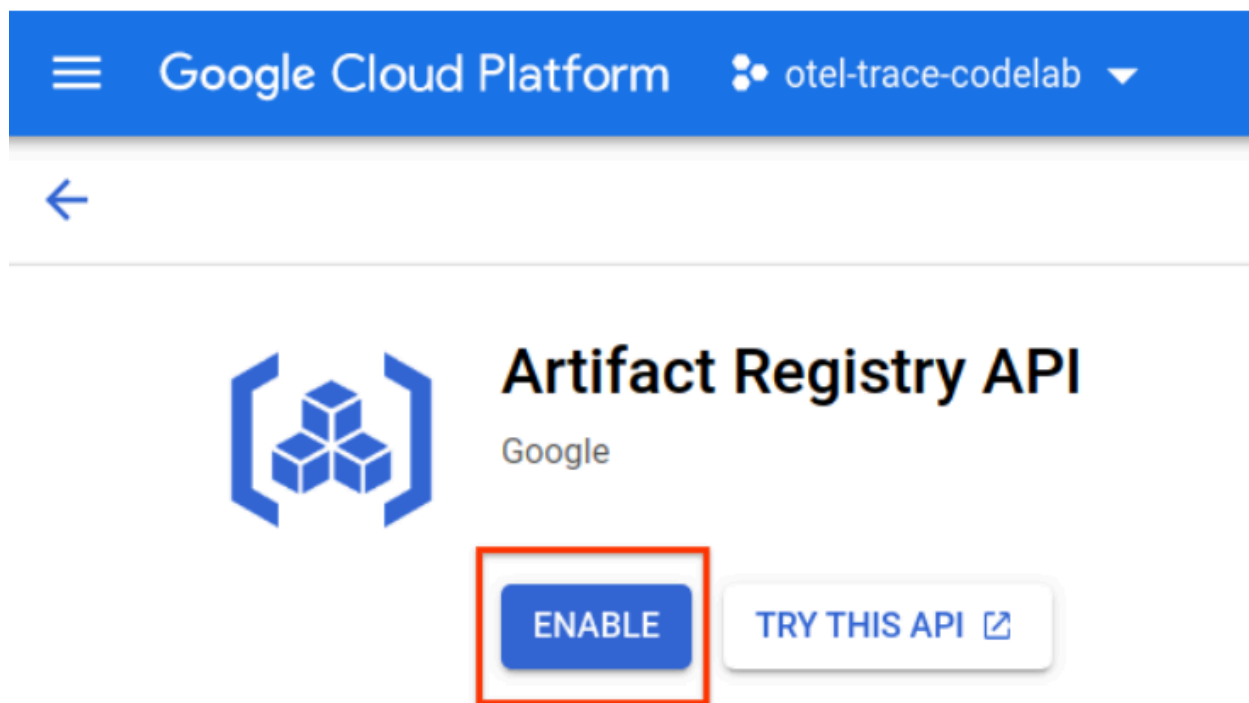
NAME	LOCATION	MASTER_VERSION	MASTER_IP	MACHINE_TYPE
otel-trace-codelab	us-central1-f	1.18.12-gke.1210	104.154.162.176	e2-medium
1.18.12-gke.1210	3	RUNNING		

設定 Artifact Registry 和 Skaffold

現在我們有可供部署的 Kubernetes 叢集。接著，我們要準備容器登錄檔，以便推送及部署容器。在這些步驟中，我們需要設定 GAR 和 skaffold 來使用。

設定 Artifact Registry

前往「Artifact Registry」選單，然後按下「啟用」按鈕。



稍待片刻，您會看到 GAR 的存放區瀏覽器。按一下「建立存放區」按鈕，然後輸入存放區名稱。

 Artifact Registry

Repositories

[+ CREATE REPOSITORY](#)

 DELETE

[SETUP INSTRUCTIONS](#)



AR Docker repositories have pricing associated with them. [Please see the pricing details in our documentation.](#) Maven and npm repositories

[DISMISS](#) [GO TO DOCUMENTATION](#)

 Filter

Enter property name or value

在本程式碼研究室中，我將新存放區命名為 `trace-codelab`。構件格式為「Docker」，位置類型為「區域」。選擇的區域應與 Google Compute Engine 預設區域相近。舉例來說，上方的範例選擇了「us-central1-f」，因此這裡我們選擇「us-central1 (Iowa)」。然後按一下「建立」按鈕。

Name *
trace-codelab

Format

☒ Docker

☐ Maven ALPHA

☐ npm ALPHA

☐ APT ALPHA

Location type

☒ Region

☐ Multi-region

Region *
us-central1 (Iowa)

Description

Labels

+ ADD LABEL

Encryption

☒ Google-managed encryption key
No configuration required

☐ Customer-managed encryption key (CMEK)
Manage via Google Cloud Key Management Service

CREATE **CANCEL**

現在，您會在存放區瀏覽器中看到「trace-codelab」。

Filter Enter property name or value				
☐	Name ↑	Format	Location	Description
☐	 trace-codelab	 Docker	us-central1 (Iowa)	

我們稍後會回到這裡檢查登錄路徑。

Skaffold 設定

Skaffold 是實用的工具，可協助您建構在 Kubernetes 上執行的微服務。只要使用一小組指令，即可處理建構、推送及部署應用程式容器的工作流程。Skaffold 預設會使用 Docker Registry 做為容器登錄檔，因此您需要設定 Skaffold，讓它在將容器推送至 GAR 時能夠辨識。

再次開啟 Cloud Shell，確認是否已安裝 Skaffold。(Cloud Shell 預設會將 skaffold 安裝至環境中)。執行下列指令，查看 Skaffold 版本。

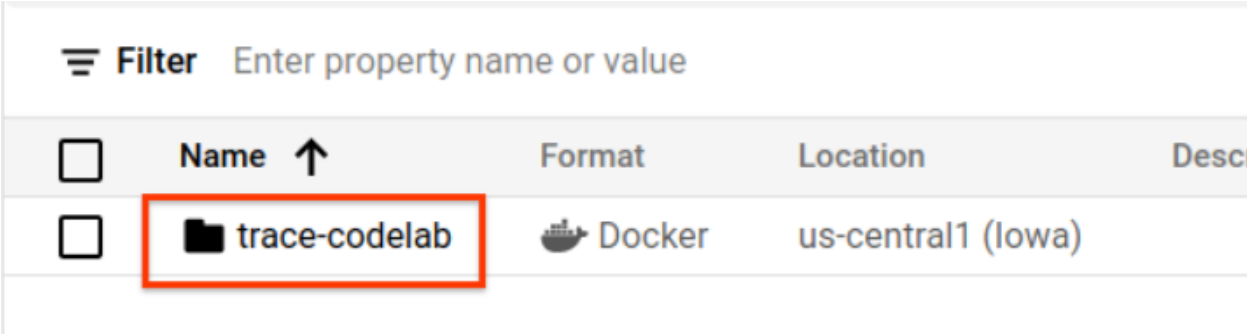
```
skaffold version
```

指令輸出

```
v1.20.0
```

注意：如果您是在 Cloud Shell 以外的環境中執行 Codelabs，請自行安裝 Skaffold。`gcloud components install` 可讓您安裝 Skaffold。

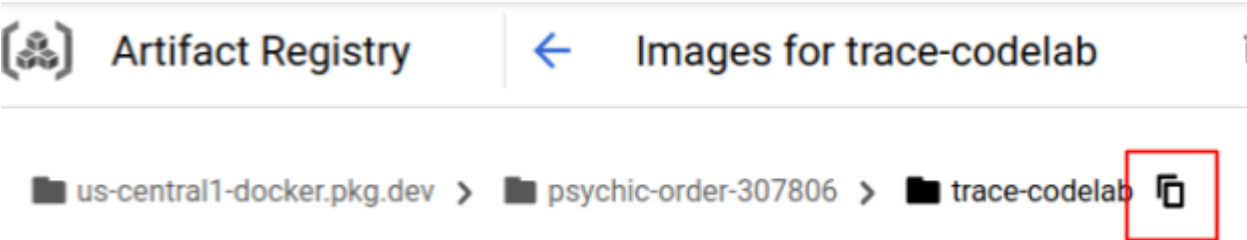
現在，您可以註冊供 Skaffold 使用的預設存放區。如要取得登錄路徑，請前往 Artifact Registry 資訊主頁，然後點選您在上一個步驟中設定的存放區名稱。



頁面頂端就會顯示導覽標記路徑。按一下



圖示，將登錄檔路徑複製到剪貼簿。



按一下複製按鈕後，瀏覽器底部會顯示對話方塊，並顯示類似下列內容的訊息：

已複製「us-central1-docker.pkg.dev/psychic-order-307806/trace-codelab」

返回 Cloud Shell。執行 `scaffold config set default-repo` 指令，並使用您剛才從資訊主頁複製的值。

```
scaffold config set default-repo us-central1-docker.pkg.dev/psychic-order-307806/trace-codelab
```

指令輸出

```
set value default-repo to us-central1-docker.pkg.dev/psychic-order-307806/trace-codelab for context gke_stackdriver-sandbox-3438851889_us-central1-b_stackdriver-sandbox
```

此外，您也需要將登錄檔設定為 Docker 設定。執行下列指令：

```
gcloud auth configure-docker us-central1-docker.pkg.dev --quiet
```

指令輸出

```
{
  "credHelpers": {
    "gcr.io": "gcloud",
    "us.gcr.io": "gcloud",
    "eu.gcr.io": "gcloud",
    "asia.gcr.io": "gcloud",
    "staging-k8s.gcr.io": "gcloud",
    "marketplace.gcr.io": "gcloud",
    "us-central1-docker.pkg.dev": "gcloud"
  }
}
Adding credentials for: us-central1-docker.pkg.dev
```

您現在可以進行下一個步驟，在 GKE 上設定 Kubernetes 容器。

摘要

在這個步驟中，您會設定程式碼研究室環境：

- 設定 Cloud Shell
- 為容器登錄建立 Artifact Registry 存放區
- 設定 Scaffold 以使用 Container Registry
- 建立 Kubernetes 叢集，讓 Codelab 微服務在其中執行

下一步

在下一個步驟中，您將建構、推送微服務，並部署到叢集

3. 建構、推送及部署微服務

下載程式碼研究室教材

在上一個步驟中，我們已設定本程式碼研究室的所有必要條件。現在您可以在這些服務上執行整個微服務。程式碼研究室資料位於 GitHub，因此請使用下列 Git 指令將資料下載至 Cloud Shell 環境。

```
cd ~
git clone https://github.com/GoogleCloudPlatform/opentelemetry-trace-codelab-python.git
```

專案的目錄結構如下：

```
shakesapp-python
├── LICENSE
├── manifests
│   ├── client.yaml
│   ├── loadgen.yaml
│   └── server.yaml
├── proto
│   └── shakesapp.proto
├── skaffold.yaml
└── src
    ├── client
    ├── loadgen
    └── server
```

- 資訊清單：Kubernetes 資訊清單檔案
- proto：用戶端與伺服器之間通訊的 proto 定義
- src：每個服務的原始碼目錄
- skaffold.yaml：Skaffold 的設定檔

執行 skaffold 指令

最後，您已準備好建構、推送及部署整個內容至剛建立的 Kubernetes 叢集。這聽起來包含多個步驟，但實際上，Skaffold 會為您完成所有作業。請使用下列指令試試看：

```
cd shakesapp-python
skaffold run --tail
```

執行指令後，您會看到 `docker build` 的記錄輸出內容，並確認這些內容已成功推送至登錄檔。

指令輸出

```
...
---> Running in c39b3ea8692b
---> 90932a583ab6
Successfully built 90932a583ab6
Successfully tagged us-centrall-docker.pkg.dev/psychic-order-307806/trace-
codelab/serverservice:step1
The push refers to repository [us-centrall-docker.pkg.dev/psychic-order-307806/trace-
codelab/serverservice]
cc8f5a05df4a: Preparing
5bf719419ee2: Preparing
2901929ad341: Preparing
88d9943798ba: Preparing
b0fdf826a39a: Preparing
3c9c1e0b1647: Preparing
f3427ce9393d: Preparing
14alca976738: Preparing
f3427ce9393d: Waiting
14alca976738: Waiting
3c9c1e0b1647: Waiting
b0fdf826a39a: Layer already exists
88d9943798ba: Layer already exists
f3427ce9393d: Layer already exists
3c9c1e0b1647: Layer already exists
14alca976738: Layer already exists
2901929ad341: Pushed
5bf719419ee2: Pushed
cc8f5a05df4a: Pushed
step1: digest:
sha256:8acdbe3a453001f120fb22c11c4f6d64c2451347732f4f271d746c2e4d193bbe size: 2001
```

推送所有服務容器後，Kubernetes 部署作業會自動啟動。

指令輸出

```

sha256:b71fce0a96cea08075dc20758ae561cf78c83ff656b04d211ffa00cedb77edf8 size: 1997
Tags used in deployment:
- serverservice -> us-central1-docker.pkg.dev/psychic-order-307806/trace-
codelab/serverservice:step4@sha256:8acdbe3a453001f120fb22c11c4f6d64c2451347732f4f271d
746c2e4d193bbe
- clientservice -> us-central1-docker.pkg.dev/psychic-order-307806/trace-
codelab/clientservice:step4@sha256:b71fce0a96cea08075dc20758ae561cf78c83ff656b04d211f
fa00cedb77edf8
- loadgen -> us-central1-docker.pkg.dev/psychic-order-307806/trace-
codelab/loadgen:step4@sha256:eea2e5bc8463ecf886f958a86906cab896e9e2e380a0eb143deaeaca
40f7888a
Starting deploy...
- deployment.apps/clientservice created
- service/clientservice created
- deployment.apps/loadgen created
- deployment.apps/serverservice created
- service/serverservice created

```

注意：如果收到「No push access to specified image repository」（無法將映像檔推送至指定存放區）等錯誤訊息，請檢查 scaffold 指令是否嘗試將映像檔推送至 Docker Hub (docker.io)，而忽略 scaffold 中預設存放區的設定。在這種情況下，請嘗試將「-default-repo」選項新增至「scaffold run」，如下所示。

```
$ scaffold run --tail --default-repo=us-central1-docker.pkg.dev/[專案 ID]/[存放區名稱]
```

部署完成後，您會在每個容器中看到實際應用程式記錄，這些記錄會發出至 stdout，如下所示：

指令輸出

```

[server] {"event": "starting server: 0.0.0.0:5050", "severity": "info", "timestamp":
"2021-03-17T05:25:56.758575Z"}
[client] [2021-03-17 05:25:54 +0000] [1] [INFO] Starting gunicorn 20.0.4
[client] [2021-03-17 05:25:54 +0000] [1] [INFO] Listening at: http://0.0.0.0:8080 (1)
[client] [2021-03-17 05:25:54 +0000] [1] [INFO] Using worker: threads
[client] [2021-03-17 05:25:54 +0000] [7] [INFO] Booting worker with pid: 7
[client] {"event": "server address is serverservice:5050", "severity": "info",
"timestamp": "2021-03-17T05:25:54.888627Z"}
[client] {"event": "request to server with query: world", "severity": "info",
"timestamp": "2021-03-17T05:26:11.550923Z"}
[server] {"event": "query: world", "severity": "info", "timestamp": "2021-03-
17T05:26:11.567048Z"}
[loadgen] {"event": "check connectivity: http://clientservice:8080/_healthz",
"severity": "info", "timestamp": "2021-03-17T05:26:11.533605Z"}
[loadgen] {"event": "/_healthz response: ok", "severity": "info", "timestamp": "2021-
03-17T05:26:11.544267Z"}
[loadgen] {"event": "confirmed connection ot clientservice", "severity": "info",
"timestamp": "2021-03-17T05:26:11.544527Z"}

```

最後，您就可以開始使用 OpenTelemetry 檢測應用程式，進行服務的分散式追蹤。

摘要

在這個步驟中，您已在環境中準備好程式碼研究室教材，並確認 Skaffold 正常運作。

下一步

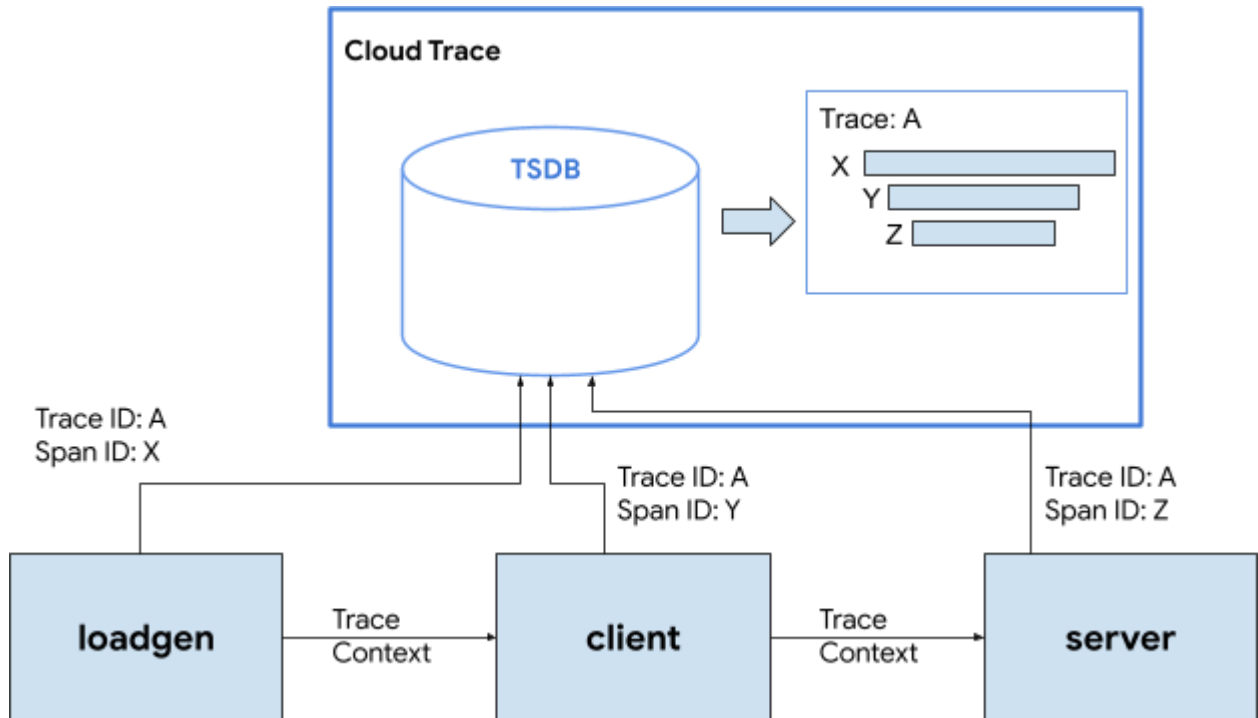
在下一個步驟中，您將修改 loadgen 服務的原始碼，以檢測追蹤記錄資訊。

步驟 4 · 約 0 分鐘

4. HTTP 的檢測

追蹤檢測和傳播的概念

編輯原始碼前，請先參閱簡單的圖表，瞭解分散式追蹤的運作方式。



在本範例中，我們會檢測程式碼，將「追蹤記錄」和「時距」資訊匯出至 Cloud Trace，並在從 loadgen 服務到伺服器服務的要求中傳播追蹤記錄結構定義。

應用程式需要傳送追蹤記錄中繼資料 (例如追蹤記錄 ID 和時距 ID)，Cloud Trace 才能將具有相同追蹤記錄 ID 的所有時距組合成一筆追蹤記錄。此外，應用程式也需要在要求下游服務時傳播追蹤脈絡 (上層時距的追蹤 ID 和時距 ID 組合)，以便下游服務瞭解要處理的追蹤脈絡。

OpenTelemetry 可協助您：

- 產生專屬的追蹤記錄 ID 和範圍 ID
- 將追蹤記錄 ID 和時距 ID 匯出至後端
- 將追蹤內容傳播至其他服務

先檢測第一個範圍

檢測負載產生器服務

按一下 Cloud Shell 右上方的

 Open Editor

按鈕，開啟 Cloud Shell 編輯器。從左側窗格的檔案總管開啟 `src/loadgen/loadgen.py`，然後找到 `main` 函式。

[src/loadgen/loadgen.py](#)

```
def main():
    ...
    # start request loop to client service
    logger.info("start client request loop")
    addr = f"http://{target}"
    while True:
        logger.info("start request to client")
        call_client(addr)
        logger.info("end request to client")
        time.sleep(2.0)
```

在 `main` 函式中，您會看到迴圈呼叫其中的 `call_client` 函式。在目前的實作中，這個區段有 2 行記錄，分別記錄函式呼叫的開始和結束。現在，我們來檢測 Span 資訊，追蹤函式呼叫的延遲時間。

首先，您需要建立具有專屬追蹤記錄 ID 和時距 ID 的時距。OpenTelemetry 提供實用的程式庫。新增下列幾行，將 OpenTelemetry 程式庫匯入程式碼。

```
import structlog
+from opentelemetry import propagate, trace
+from opentelemetry.exporter.cloud_trace import CloudTraceSpanExporter
+from opentelemetry.sdk.trace import TracerProvider
+from opentelemetry.instrumentation.requests import RequestsInstrumentor
+from opentelemetry.sdk.trace.export import SimpleSpanProcessor
+from opentelemetry.propagators.cloud_trace_propagator import CloudTraceFormatPropagator
```

由於負載產生器會透過 `requests` 模組以 HTTP 呼叫用戶端應用程式，因此我們使用 `requests` 的擴充套件，並啟用檢測。

```
from opentelemetry.propagators.cloud_trace_propagator import CloudTraceFormatPropagator
+
+RequestsInstrumentor().instrument()
```

接著設定 Tracer 執行個體，處理 Trace Context 和匯出工具設定

```
target = os.environ.get("CLIENT_ADDR", "0.0.0.0:8080")

+ exporter = CloudTraceSpanExporter()
+ trace.get_tracer_provider().add_span_processor(SimpleSpanProcessor(exporter))
+ tracer = trace.get_tracer(__name__)
+ propagate.set_global_textmap(CloudTraceFormatPropagator())
+ trace.set_tracer_provider(TracerProvider())
+
    # connectivity check to client service
    healthz = f"http://{target}/_healthz"
    logger.info(f"check connectivity: {healthz}")
```

請注意，本程式碼研究室旨在瞭解追蹤檢測的運作方式，因此我們會將 Tracer 設為記錄每項要求，並將要求傳送至後端。`(SimpleSpanProcessor())` 這不適合用於正式環境，因此請務必在檢測正式版應用程式時變更這部分。

您現在可以使用追蹤器，為 Span 加上儀表。重點在於您需要明確產生 Span，就這樣！雖然有兩行程式碼會將事件中繼資料新增至 Span，但您不需要手動產生不重複的追蹤 ID 和 Span ID，並將其嵌入 Span。

```
logger.info("start client request loop")
addr = f"http://{target}"
while True:
    logger.info("start request to client")
    call_client(addr)
    logger.info("end request to client")
    with tracer.start_as_current_span("loadgen") as root_span:
        root_span.add_event(name="request_start")
        logger.info("start request to client")
        call_client(addr)
        root_span.add_event(name="request_end")
        logger.info("end request to client")
    time.sleep(2.0)
```

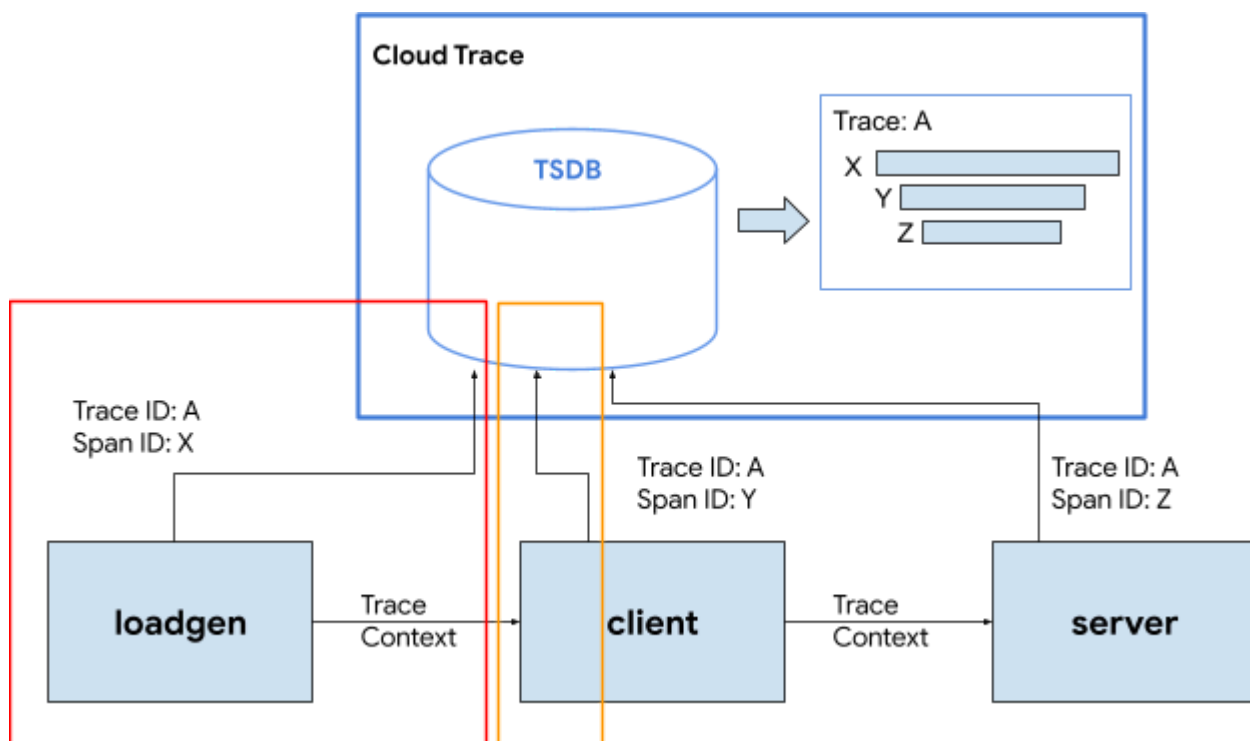
如要讓 Docker 建構作業擷取必要的 OpenTelemetry 套件，請執行下列指令：

```
poetry add "opentelemetry-exporter-gcp-trace=^1.0.0rc0"
poetry add "opentelemetry-propagator-gcp=^1.0.0rc0"
poetry add "opentelemetry-instrumentation-requests=^0.20b0"
```

您可以確認相應的依附元件說明是否以 `pyproject.toml` 撰寫。

檢測用戶端服務

在上一節中，我們對下圖中以紅色矩形框住的部分進行了插樁。我們在負載產生器服務中，設定了範圍資訊。與負載產生器服務類似，現在我們需要檢測用戶端服務。與負載產生器服務不同的是，用戶端服務需要從 HTTP 標頭中擷取負載產生器服務傳播的追蹤記錄 ID 資訊，並使用該 ID 產生範圍。



開啟 Cloud Shell 編輯器，然後新增必要模組，就像我們為負載產生器服務所做的一樣。

src/client/client.py

```
import flask
import grpc
import structlog
+from opentelemetry import propagate, trace
+from opentelemetry.exporter.cloud_trace import CloudTraceSpanExporter
+from opentelemetry.instrumentation.flask import FlaskInstrumentor
+from opentelemetry.sdk.trace import TracerProvider
+from opentelemetry.sdk.trace.export import SimpleSpanProcessor
+from opentelemetry.propagators.cloud_trace_propagator import \
+    CloudTraceFormatPropagator

import shakesapp_pb2
import shakesapp_pb2_grpc
```

您發現自己剛匯入 `FlaskInstrumentor`，這項功能可代表使用者自動檢測 Flask 應用程式，並擷取 HTTP 標頭，以單行程式碼取得追蹤內容。OpenTelemetry 社群也提供類似的實用整合功能，可與其他主要程式庫搭配使用。詳情請參閱[官方說明文件](#)。

```
app = flask.Flask(__name__)
+FlaskInstrumentor().instrument_app(app)
```

開始檢測前，您需要再次準備 Tracer 執行個體，做法與負載產生器服務中的做法類似。

```
logger.info(f"server address is {SERVER_ADDR}")

+exporter = CloudTraceSpanExporter()
+trace.get_tracer_provider().add_span_processor(SimpleSpanProcessor(exporter))
+propagate.set_global_textmap(CloudTraceFormatPropagator())
+trace.set_tracer_provider(TracerProvider())

@app.route("/")
def main_handler():
    ....
```

現在可以準備在處理常式中新增檢測設備。找出 `main_handler()`，並修改向伺服器服務擲回 gRPC 要求的程式碼部分。

```

@app.route("/")
def main_handler():
    q, count = random.choice(list(queries.items()))

    # get Tracer
    tracer = trace.get_tracer(__name__)

    with tracer.start_as_current_span("client") as cur_span:
        channel = grpc.insecure_channel(SERVER_ADDR)
        stub = shakesapp_pb2_grpc.ShakespeareServiceStub(channel)
        logger.info(f"request to server with query: {q}")
        cur_span.add_event("server_call_start")
        resp = stub.GetMatchCount(shakesapp_pb2.ShakespeareRequest(query=q))
        cur_span.add_event("server_call_end")
        if count != resp.match_count:
            raise UnexpectedResultError(
                f"The expected count for '{q}' was {count}, but result was {resp.match_count}
            } obtained"
            )
        result = str(resp.match_count)
        logger.info(f"matched count for '{q}' is {result}")
    return result

```

與負載產生器服務類似，使用下列指令將必要套件新增至 `pyproject.toml`。

```

poetry add "opentelemetry-exporter-gcp-trace=^1.0.0rc0"
poetry add "opentelemetry-propagator-gcp=^1.0.0rc0"
poetry add "opentelemetry-instrumentation-flask=^0.20b0"

```

接著，請嘗試使用 `scaffold run` 指令啟動應用程式，並查看 Cloud Trace 資訊主頁顯示的內容：

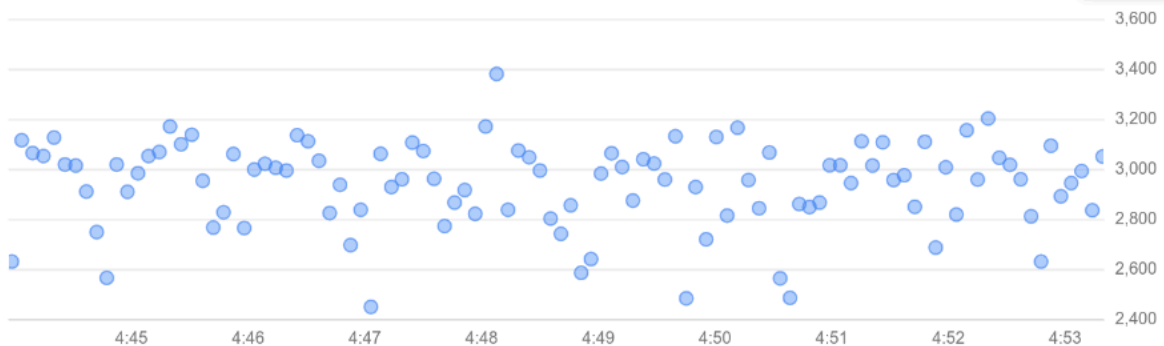
```
scaffold run --tail
```

看到一些建構、推送及部署訊息後，您會看到 JSON 格式的應用程式記錄。前往 Cloud Trace > 追蹤記錄清單，確認是否取得追蹤記錄資訊。由於負載產生器服務會定期將要求傳送至用戶端服務，且您已為所有要求啟用追蹤記錄，因此追蹤記錄清單上會開始顯示許多點。

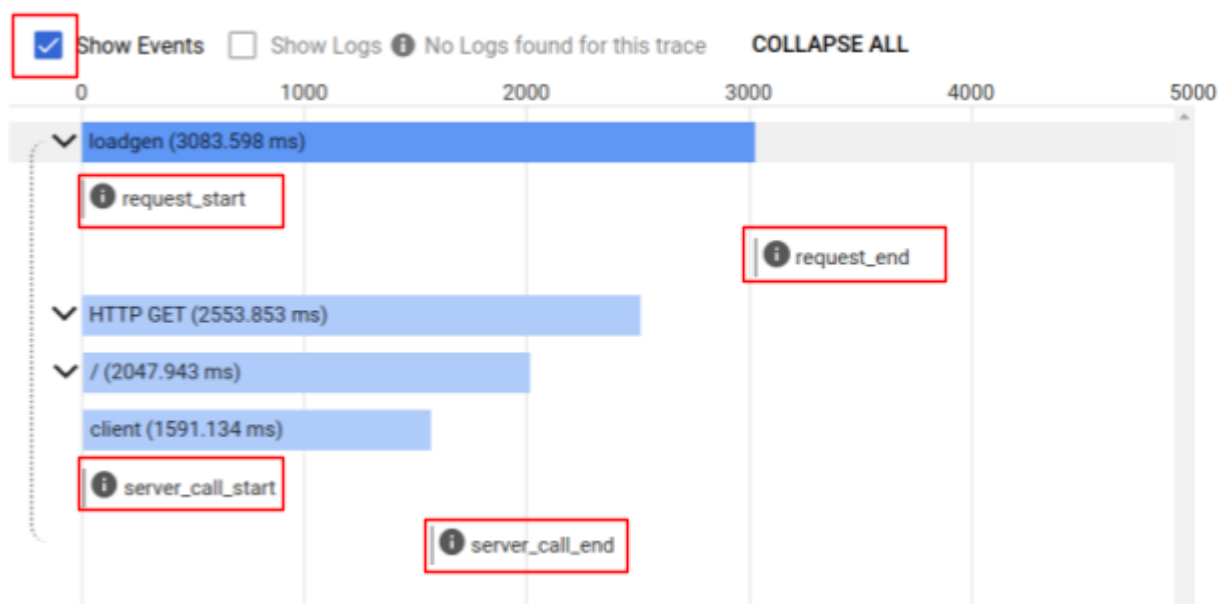
Filter Add trace filter...

Select a trace

RESET



點選其中一個，您會看到類似下方的瀑布圖，說明要求和回應程序中每個部分的延遲時間。找到「顯示事件」旁的核取方塊，勾選後瀑布圖中就會顯示註解。這些註解是您透過 `span.add_event()` 方法在程式碼中插建的註解。



您可能會發現伺服器服務的範圍並未顯示。這是正確的，因為我們完全沒有在伺服器服務中檢測 Span。

摘要

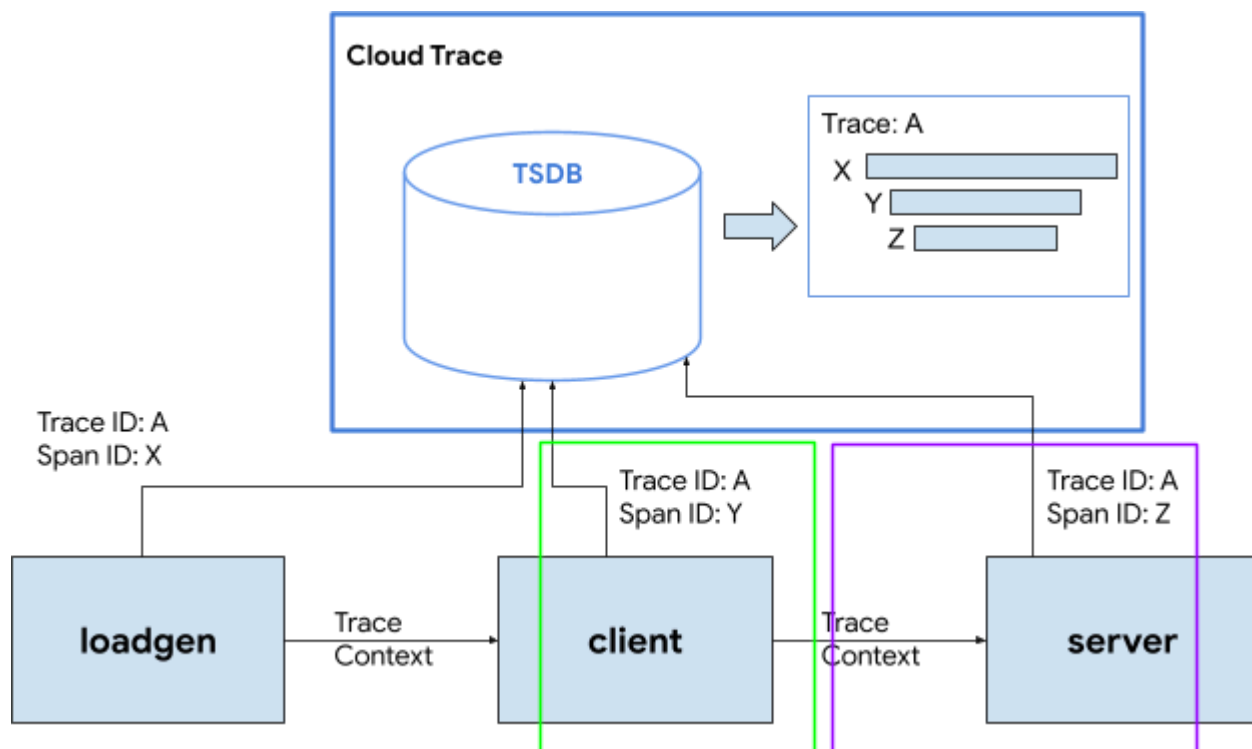
在本步驟中，您已檢測負載產生器服務和用戶端服務，並確認您可以在服務之間成功傳播追蹤記錄內容，以及將這兩項服務的範圍資訊匯出至 Cloud Trace。

下一步

在下一個步驟中，您將檢測用戶端服務和伺服器服務，確認如何透過 gRPC 傳播追蹤內容。

5. gRPC 的檢測

在上一個步驟中，我們在這個微服務中檢測了要求的前半部分。在本步驟中，我們會嘗試在用戶端服務和伺服器服務之間，檢測 gRPC 通訊。(下圖中的綠色和紫色矩形)



自動檢測 gRPC 用戶端

OpenTelemetry 生態系統提供許多實用的程式庫，可協助開發人員檢測應用程式。在上一個步驟中，我們對「要求」模組使用了自動檢測功能。在這個步驟中，由於我們嘗試透過 gRPC 傳播追蹤記錄內容，因此使用相關程式庫。

[src/client/client.py](#)

```
import flask
import grpc
import structlog
from opentelemetry import propagate, trace
from opentelemetry.exporter.cloud_trace import CloudTraceSpanExporter
from opentelemetry.instrumentation.flask import FlaskInstrumentor
+from opentelemetry.instrumentation.grpc import GrpcInstrumentorClient
from opentelemetry.sdk.trace import TracerProvider
from opentelemetry.sdk.trace.export import SimpleSpanProcessor
from opentelemetry.propagators.cloud_trace_propagator import \
    CloudTraceFormatPropagator
import shakespeare_pb2
import shakespeare_pb2_grpc

app = flask.Flask(__name__)
FlaskInstrumentor().instrument_app(app)
+GrpcInstrumentorClient().instrument()
```

就用戶端服務而言，我們需要執行的檢測作業相當少。我們想透過 gRPC 傳播「追蹤記錄內容」，也就是目前時距的追蹤記錄 ID 和時距 ID 組合。因此我們呼叫 `GrpcInstrumentatorClient.instrument()`，讓處理常式函式中的 gRPC 用戶端將追蹤情境嵌入下方的 HTTP 標頭。

請務必使用 `poetry add` 指令，將新的依附元件新增至 `pyproject.toml`：

```
poetry add "opentelemetry-instrumentation-grpc" ^0.20b0
```

自動檢測 gRPC 伺服器

與 gRPC 用戶端相同，我們也會呼叫 gRPC 伺服器的自動檢測功能。在檔案頂端新增下列匯入項目，並呼叫 `GrpcInstrumentationServer().instrument()`。

注意：請務必撥打

```
GrpcInstrumentationServe()
```

在這個步驟中，

```
GrpcInstrumentationClient()
```

。

[src/server/server.py](#)

```
import grpc
import structlog
from google.cloud import storage
from grpc_health.v1 import health_pb2, health_pb2_grpc
+from opentelemetry import propagate, trace
+from opentelemetry.exporter.cloud_trace import CloudTraceSpanExporter
+from opentelemetry.instrumentation.grpc import GrpcInstrumentorServer
+from opentelemetry.sdk.trace import TracerProvider
+from opentelemetry.sdk.trace.export import SimpleSpanProcessor
+from opentelemetry.propagators.cloud_trace_propagator import CloudTraceFormatPropagator

import shakesapp_pb2
import shakesapp_pb2_grpc

BUCKET_NAME = "dataflow-samples"
BUCKET_PREFIX = "shakespeare/"

+# enable auto gRPC server trace instrumentation
+GrpcInstrumentorServer().instrument()
+
```

接著，您會新增匯出工具，將追蹤資訊傳送至 Cloud Trace 後端。在 `serve()` 函式中新增下列程式碼。

```
def serve():
+   # start trace exporter
+   trace.set_tracer_provider(TracerProvider())
+   trace.get_tracer_provider().add_span_processor(
+       SimpleSpanProcessor(CloudTraceSpanExporter())
+   )
+   propagators.set_global_textmap(CloudTraceFormatPropagator())
+
+   # add gRPC services to server
  server = grpc.server(futures.ThreadPoolExecutor(max_workers=4))
  service = ShakesappService()
  shakesapp_pb2_grpc.add_ShakespeareServiceServicer_to_server(service, server)
  health_pb2_grpc.add_HealthServicer_to_server(service, server)
```

請務必在伺服器服務中新增套件。

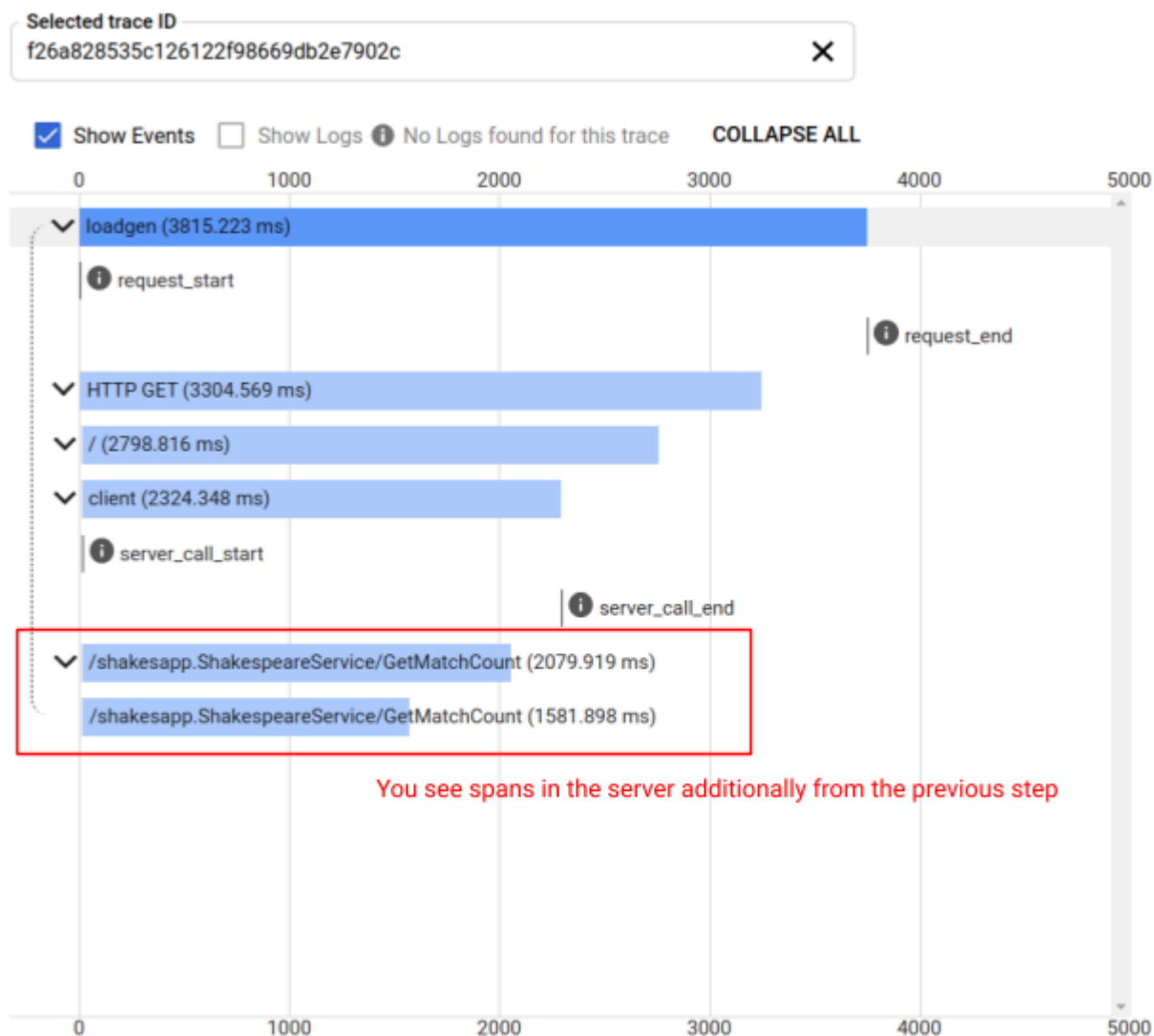
```
poetry add "opentelemetry-exporter-gcp-trace=^1.0.0rc0"
poetry add "opentelemetry-instrumentation-grpc=^0.20b0"
poetry add "opentelemetry-propagator-gcp=^1.0.0rc0"
poetry add "opentelemetry-instrumentation=^0.20b0"
```

執行微服務並確認追蹤記錄

然後使用 `skaffold` 指令執行修改後的程式碼。

```
skaffold run --tail
```

現在您又會在 Cloud Trace 的「Trace list」(追蹤記錄清單) 頁面中看到一堆追蹤記錄。按一下其中一個追蹤記錄，您會發現要求跨越負載產生器服務到伺服器服務。



摘要

在這個步驟中，您使用 OpenTelemetry 生態系統程式庫的支援，檢測了以 gRPC 為基礎的通訊。此外，您也確認負載產生器服務中產生的追蹤記錄內容已成功傳送至伺服器服務。

6. 恭喜

您已成功使用 OpenTelemetry 建立分散式追蹤記錄，並在 Google Cloud Trace 中確認微服務的要求延遲時間。

如要進行延伸練習，可以自行嘗試下列主題。

- 目前的實作方式會傳送健康狀態檢查產生的所有 span。如何從 Cloud Trace 篩除這些時距？提示請見[這裡](#)。
- 將事件記錄檔與時距相互關聯，並瞭解這項功能在 Google Cloud Trace 和 Google Cloud Logging 中的運作方式。提示請見[這裡](#)。
- 以其他語言的服務取代部分服務，然後嘗試使用該語言的 OpenTelemetry 進行檢測

注意：Google Kubernetes Engine 和 Google Artifact Registry 會持續消耗資源。

清理

完成本程式碼研究室後，請停止 Kubernetes 叢集，並務必刪除專案，以免 Google Kubernetes Engine、Google Cloud Trace 和 Google Artifact Registry 產生非預期的費用。

首先，請使用下列指令刪除叢集：

```
skaffold delete
```

指令輸出

```
Cleaning up...
- deployment.apps "clientservice" deleted
- service "clientservice" deleted
- deployment.apps "loadgen" deleted
- deployment.apps "serverservice" deleted
- service "serverservice" deleted
```

刪除叢集後，請依序選取選單窗格中的「IAM & Admin」>「Settings」，然後按一下「SHUT DOWN」按鈕。

Settings

Project name
otel-trace-codelab SAVE

Project ID
psychic-order-307806

Project number:
850914116791

然後在對話方塊的表單中輸入專案 ID (而非專案名稱)，並確認要關閉專案。
